

15 AI Engineering Projects by Kushal Vijay

A Step-by-Step Build Guide to AI Engineering Roles

Best Fit for Mid level Engineers, AI Enthusiasts, College Students

Complete tutorial coming soon on <https://www.youtube.com/@KushalVijay/>

How to Use:

This guide gives you fifteen portfolio project blueprints for software engineers who already know how to build APIs, ship services, debug production issues, and work with databases. The goal is to show those same skills in AI systems.

The projects are not meant to be copied line by line from a tutorial. Treat them like product specs. You'll make architecture choices, run into tradeoffs, write tests, and collect metrics. Those decisions are what you'll talk about in interviews.

Why you should build these Projects

- Every project solves a practical problem teams face after the first AI demo works: quality, cost, evals, latency, observability, safety, deployment, and rollout risk.
- Each project turns your existing backend engineering experience into an advantage instead of asking you to become a research scientist overnight.
- The focus is not "I called an LLM API." The focus is "I built the system around the model that makes it usable in production."
- Each project includes a topic label so you can pick based on the roles you're targeting: RAG, LLMOps, MLOps, agents, guardrails, evals, multimodal, or deployment.

Project Topic Map

Project	Project Name	Primary Topic	Secondary Topics
1	Support Knowledge Copilot with Verified Citations	RAG	Hybrid retrieval, citation verification, evals
2	Prompt Release Safety Gate	LLMOps	PromptOps, CI/CD, regression testing
3	LLM Spend Control Center	LLMOps	Cost optimization, routing, observability
4	AI Output Policy Guardrail Service	Guardrails	Safety, compliance, structured validation
5	Production Log-to-Eval Dataset Builder	Evals	Data flywheel, monitoring, LLM-as-judge
6	Permissioned Tool-Using Agent Sandbox	Agents	Tool use, security, human-in-the-loop
7	RAG Freshness and Drift Monitor	RAG	Data quality, monitoring, re-indexing
8	Natural Language to API Assistant	Tool Calling	Guardrails, OpenAPI, workflow automation
9	Multimodal Document Intake Reviewer	Multimodal AI	OCR, extraction, human review
10	Fine-Tune vs RAG Decision Lab	MLOps	LoRA, benchmarking, model selection
11	AI Trace Explorer for LLM Workflows	LLMOps	Observability, tracing, debugging

Project	Project Name	Primary Topic	Secondary Topics
12	Semantic Cache Gateway for LLM APIs	LLMOps	Cost reduction, latency, embeddings
13	AI Feature Rollout Monitor	LLMOps	Feature flags, canary rollout, rollback
14	Evaluation-as-a-Service Platform	Evals	Judge calibration, quality gates, analytics
15	Local LLM Deployment Kit	MLOps	Inference, quantization, performance testing

Project 1: Support Knowledge Copilot with Verified Citations

Topic: RAG, Hybrid Retrieval, Citation Verification, Retrieval Evaluation

What You're Building: A support knowledge assistant that answers employee or customer support questions using internal documentation, retrieves evidence from multiple sources, generates grounded answers with citations, and verifies whether every citation supports the claim it is attached to.

Why This Project Lands Interviews
A basic RAG demo is easy. This version gives you concrete things to discuss: chunking choices, hybrid retrieval, citation checks, no-answer handling, and eval numbers.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Strong ecosystem for AI, APIs, and data processing
API	FastAPI	Async-friendly and widely used for API services

Component	Tool / Library	Why This Choice
Embeddings	OpenAI text-embedding-3-small or bge-small	Good quality with manageable cost
Vector Store	Qdrant or ChromaDB	Easy local setup, metadata filtering, scalable path later
Sparse Search	BM25 via rank_bm25	Catches exact matches like error codes and product names
LLM	GPT-4o-mini / Claude Haiku / local Llama	Flexible answer generation layer
Evals	RAGAS, DeepEval, or custom metrics	Measures retrieval and answer quality separately
UI	Streamlit	Fast demo dashboard without frontend overhead
Containerization	Docker Compose	Makes the full stack easy to run locally

Step-by-Step Build Guide

Phase 1: Define the Knowledge Assistant Scope (Day 1-2)

- 1. Pick a realistic document set:** Use a small but believable support corpus: product FAQs, troubleshooting guides, onboarding docs, API docs, release notes, and policy pages. The corpus should have overlapping information so retrieval is not too easy.
- 2. Define the assistant contract:** Input is a user question. Output is an answer, source citations, confidence score, and a short "what I could not verify" section. This contract matters because production RAG systems need to be honest about uncertainty.
- 3. Create metadata rules:** Every document should include source name, section heading, last updated date, document type, and access level. Store this metadata from the beginning so you can filter retrieval later.

Phase 2: Build Ingestion and Chunking (Day 2-4)

- 1. Normalize documents into clean text:** Write loaders for Markdown, HTML, text files, and PDFs. Preserve headings and page numbers wherever possible. Store both raw and cleaned versions so you can debug indexing issues.
- 2. Implement multiple chunking strategies:** Start with recursive heading-based chunking, then add fixed-size chunking with overlap. Track which strategy produced each chunk so you can compare performance later.
- 3. Generate embeddings and indexes:** Embed every chunk and store it in the vector database. Build a BM25 index over the same chunk IDs. Both indexes should point to the same metadata so the fusion layer is clean.
- 4. Add a re-index command:** Build a CLI command like `python ingest.py --source docs/ --rebuild`. This keeps it closer to how a teammate would use the tool.

Interview Talking Point
Explain why you kept dense and sparse indexes over the same chunk IDs. It shows you understand that semantic search and keyword search solve different failure modes.

Phase 3: Build Hybrid Retrieval (Day 4-6)

- 1. Implement dense retrieval:** Embed the user question and retrieve the top-k closest chunks by cosine similarity. Store the raw similarity score for debugging.
- 2. Implement sparse retrieval:** Run BM25 over the same question and retrieve keyword-relevant chunks. This is especially useful for exact phrases, API names, SKUs, and error codes.
- 3. Fuse results with RRF:** Use Reciprocal Rank Fusion to merge dense and sparse result lists. Make the weights configurable so you can show how retrieval changes under different settings.
- 4. Add a reranking pass:** Rerank the top 20 fused chunks using a small cross-encoder or LLM-as-reranker. Keep the top 5 chunks for generation.

Phase 4: Build Grounded Answer Generation (Day 6-8)

- 1. Design the answer prompt:** Tell the model to answer only from provided context, cite claims using chunk IDs, and say when the answer is not available. Keep the prompt simple and strict.
- 2. Verify citations after generation:** Parse citations from the answer and check whether each cited chunk supports the claim. Use an LLM-as-judge or rule-based claim extraction for V1.
- 3. Create confidence scoring:** Combine retrieval score, citation support rate, answer completeness, and "no-answer" detection into a single confidence score. Return the breakdown along with the final number.
- 4. Handle missing knowledge gracefully:** If retrieval confidence is low, return a helpful "I could not find this in the docs" response with the closest matching sections. A real user can act on that response.

Phase 5: Build the Evaluation Suite and Dashboard (Day 8-11)

- 1. Create a golden Q&A set:** Write 50-75 questions by hand. Include simple lookups, multi-doc questions, ambiguous questions, outdated-document traps, and questions that have no answer in the corpus.
- 2. Measure retrieval and answer quality separately:** Track whether the right chunks were retrieved, whether the answer is correct, whether citations are valid, and whether the system correctly refused when the answer was missing.
- 3. Build the dashboard:** Show the question, answer, retrieved chunks, citation verdicts, and confidence breakdown. Add a toggle to compare dense-only vs. hybrid retrieval.
- 4. Add an eval command:** Create `python eval.py --strategy hybrid` and generate a Markdown or HTML report with metrics. This report is the artifact reviewers will open first.

Phase 6: Polish for Portfolio (Day 11-12)

- 1. Record a short walkthrough:** Show ingestion, a good answer with verified citations, a failed citation being caught, and a no-answer case handled correctly.
- 2. Write the case study:** Start with a measurable result like: "Hybrid retrieval improved correct-source retrieval from 72% to 88% on a 60-question eval set." Then explain the architecture and tradeoffs.

Project 2: Prompt Release Safety Gate

Topic: LLMOps, PromptOps, CI/CD, Regression Testing, Evals

What You're Building: A CI/CD safety gate for prompt changes. Whenever a prompt file changes in a pull request, the system runs a regression suite, compares the new prompt against the current production prompt, and blocks release if quality, cost, latency, or safety metrics degrade beyond a threshold.

Why This Project Lands Interviews
Interviewers care less about a clever prompt and more about how it is shipped safely. This build gives you a clean story around prompt versioning, test data, release gates, and rollback.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Simple for eval runners and automation
Prompt Format	YAML	Human-readable and easy to diff
Structured Output	Pydantic	Keeps model responses testable
LLM Provider	OpenAI / Anthropic / local model	Lets you compare providers later
Eval Runner	Custom + DeepEval	Gives flexibility and recognizable tooling
Storage	SQLite + JSON artifacts	Portable, inspectable, easy for a portfolio repo
CI/CD	GitHub Actions	Natural place to block bad prompt releases
Reporting	Markdown + HTML report	Works in PR comments and browser views

Component	Tool / Library	Why This Choice
Alerting	Slack webhook	Mirrors real team workflows

Step-by-Step Build Guide

Phase 1: Pick the AI Feature Under Test (Day 1-2)

- 1. Build a small production-like feature:** Create an AI feature such as "turn messy customer notes into a clean CRM summary." The output should be structured: summary, sentiment, next action, urgency, and confidence.
- 2. Store prompts as versioned YAML:** Put prompts in `/prompts`. Each file should include version, owner, description, model settings, system prompt, few-shot examples, and expected output schema.
- 3. Define the response contract:** Use Pydantic to validate the output. A prompt that returns beautiful prose but breaks the schema should fail the release gate.

Phase 2: Build the Golden Test Set (Day 2-4)

- 1. Hand-write realistic examples:** Create 75-100 messy notes with expected structured outputs. Include spelling mistakes, vague requests, emotional customers, missing context, and mixed-intent notes.
- 2. Label edge cases clearly:** Add fields like `difficulty`, `risk_area`, and `why_this_case_exists`. These notes make your dataset feel intentionally designed instead of randomly generated.
- 3. Version the dataset:** Store it in JSONL with stable IDs. When you update the dataset, write a changelog. This shows that the eval bar is managed too, not treated as a one-time file.

Interview Talking Point

Say that the dataset is the real product here. The CI runner is useful, but the human-curated edge cases are what make the eval meaningful.

Phase 3: Build the Regression Runner (Day 4-7)

- 1. Run baseline and candidate prompts:** For every test case, call the production prompt and the changed prompt. Store raw outputs, parsed outputs, latency, token counts, and model errors.

- 2. Score multiple dimensions:** Measure schema validity, field-level correctness, summary relevance, next-action usefulness, safety issues, latency, and cost. Do not collapse everything into one vague score too early.
- 3. Compare run-over-run:** Identify cases that passed before but fail now, cases that improved, categories that regressed, and cost/latency changes. This diff is the core value of the project.
- 4. Add thresholds:** Create warning and blocking thresholds. Example: block if schema validity drops by more than 2%, safety failures increase, or average cost rises by more than 20%.

Phase 4: Build Reports and PR Comments (Day 7-9)

- 1. Generate a release report:** Include a scorecard, regression table, examples of changed outputs, cost delta, latency delta, and recommended release decision.
- 2. Add side-by-side output diffs:** For every failed case, show input, baseline output, candidate output, expected output, and the scoring explanation.
- 3. Post a PR comment:** The GitHub Action should post a short summary: pass/warn/fail, top regressions, metric deltas, and a link to the full report artifact.

Phase 5: Wire into CI/CD (Day 9-11)

- 1. Trigger only when prompts change:** Configure GitHub Actions to run when files under `/prompts` or `/evals` change. Keep the workflow efficient.
- 2. Block risky merges:** If the result is critical, exit non-zero so the PR cannot merge. If it is warning-only, allow merge but leave a visible warning.
- 3. Package the runner:** Add a Dockerfile so the same runner works locally and in CI. Expose environment variables for API keys, thresholds, and model choice.

Phase 6: Polish for Portfolio (Day 11-12)

- 1. Create a demo PR:** Intentionally change a prompt so it gets more verbose, more expensive, or less accurate. Show the safety gate catching it.
- 2. Write your README like team documentation:** Include setup, how to add test cases, how to adjust thresholds, and what decisions you made around LLM-as-judge scoring.

Project 3: LLM Spend Control Center

Topic: LLMOps, Cost Optimization, Model Routing, Budget Monitoring

What You're Building: A routing and budgeting layer that sits in front of LLM calls, tracks usage by team or feature, chooses cost-effective models when appropriate, and warns or blocks usage when budgets are at risk.

Why This Project Lands Interviews
Quality matters, but so does the bill. This build lets you talk about routing, budgets, latency, and the tradeoff between cost and answer quality.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Easy integration with AI providers
API Gateway	FastAPI	Clean async proxy layer
Storage	PostgreSQL	Durable usage, budgets, and audit records
Cache / Counters	Redis	Fast budget checks and rate counters
Providers	OpenAI, Anthropic, Ollama	Mix of paid and local options
Classifier	scikit-learn or small LLM	Request complexity classification
Dashboard	Streamlit or Grafana	Cost visibility for humans
Worker	Celery / RQ	Async verification and reporting jobs
Containerization	Docker Compose	Gateway + database + dashboard locally

Step-by-Step Build Guide

Phase 1: Build the Unified Request Gateway (Day 1-3)

1. **Define a standard request format:** Accept chat-style input, team ID, feature name, priority, and optional model preference. Normalize all provider responses into one response schema.
2. **Create a model registry:** Store model name, provider, quality tier, input cost, output cost, latency estimate, max context length, and supported features like vision or tool calling.
3. **Build provider adapters:** Implement one adapter per provider. Each adapter should return output text, token counts, latency, cost, and provider metadata.

Phase 2: Build Cost Tracking and Budgets (Day 3-5)

1. **Log every request:** Store timestamp, team ID, feature, model used, input/output tokens, latency, status, and cost. Make this audit trail queryable.
2. **Add budget policies:** Each team or feature gets daily and monthly limits. Track spend against those limits in real time.
3. **Create warning and block behavior:** At 80% budget, send warning alerts. At 100%, block low-priority requests or require override. Return clear errors instead of silently failing.

Interview Talking Point
Explain that budgets should be enforced before the provider call, but final cost is only known after the response. That small detail shows real systems thinking.

Phase 3: Build Request Complexity Routing (Day 5-8)

1. **Define routing tiers:** Tier 1 is extraction and formatting. Tier 2 is summarization and classification. Tier 3 is reasoning-heavy or high-risk work.
2. **Build a lightweight classifier:** Use features like prompt length, instruction verbs, required output format, context size, and risk tags. A simple model is fine; the architecture matters more than perfect ML.
3. **Map tiers to models:** Route simple work to cheaper models, moderate work to mid-tier models, and risky work to high-quality models. Store this mapping in YAML or database config.
4. **Add override rules:** Some features should always use a stronger model because correctness matters more than cost. Make those rules explicit.

Phase 4: Add Quality Checks and Escalation (Day 8-10)

1. **Sample responses for verification:** For a percentage of requests routed to cheaper models, asynchronously compare output quality against a stronger model.
2. **Detect bad routing decisions:** If the cheap model fails a quality check, mark the request as a routing miss. Store the prompt, chosen model, better model, and reason.
3. **Add auto-escalation for high-risk requests:** If confidence is low or the request is tagged high-priority, rerun with a stronger model before returning the final answer.

Phase 5: Build the Cost Dashboard (Day 10-12)

1. **Show spend by team and feature:** Include daily cost, monthly projection, top expensive prompts, and cost by model.
2. **Show savings estimates:** Compare actual routed spend with "everything sent to the strongest model." This gives you the main metric for the case study.
3. **Add routing quality metrics:** Show escalation rate, verifier pass rate, latency by model, and error rate by provider.

Phase 6: Polish for Portfolio (Day 12-14)

1. **Run a simulated workload:** Send 1,000 mixed prompts through the gateway and produce a cost savings report.
2. **Write the case study:** Start with: "Reduced simulated LLM spend by X% while maintaining Y% verification pass rate." Then show the routing design and budget enforcement flow.

Project 4: AI Output Policy Guardrail Service

Topic: Guardrails, Safety, Compliance, Structured Validation

What You're Building: A standalone guardrail service that reviews LLM outputs before users see them. It checks for policy violations, unsupported claims, PII leakage, unsafe instructions, schema errors, and brand-tone issues, then either approves, rewrites, blocks, or routes the output for review.

Why This Project Lands Interviews
Teams need AI outputs they can defend. This build gives you a practical safety layer with policies, structured checks, review queues, and audit logs.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Strong validation and API ecosystem
API	FastAPI	Easy to integrate as middleware
Rules Engine	Custom YAML policy rules	Transparent and auditable
Validation	Pydantic + regex + classifiers	Combines deterministic and AI checks
LLM Judge	GPT-4o-mini / Claude Haiku	Reviews nuanced policy issues
PII Detection	Presidio or custom detectors	Practical privacy protection
Storage	PostgreSQL	Policy versions, decisions, audit logs
UI	Streamlit	Quick review and audit dashboard
Containerization	Docker	Easy deployment as a sidecar service

Step-by-Step Build Guide

Phase 1: Define Policies and Decision Types (Day 1-2)

- 1. Create policy categories:** Start with PII leakage, unsupported factual claims, toxic language, unsafe instructions, medical/legal/financial overconfidence, schema mismatch, and brand voice violations.
- 2. Define decision outcomes:** The guardrail can approve, approve with warning, rewrite, block, or send to human review. Keep these outcomes simple and explainable.

3. Build policy files: Store policies in YAML with rule name, severity, examples, detection strategy, and recommended action. This keeps the service auditable.

Phase 2: Build Deterministic Validators (Day 2-5)

1. Add schema validation: If an AI feature expects JSON, validate it before anything else. Bad format is the easiest failure to catch and the most common to ignore.

2. Add PII detection: Detect emails, phone numbers, addresses, API keys, credit card-like strings, and named entities. Allow per-feature configuration because not all PII is always forbidden.

3. Add forbidden content checks: Use regex and keyword rules for obvious policy violations. Deterministic checks should catch cheap, clear failures before using an LLM judge.

Interview Talking Point
Emphasize that not every guardrail should be an LLM call. Cheap deterministic checks reduce latency, cost, and false explanations.

Phase 3: Build LLM-Based Policy Review (Day 5-7)

1. Create judge prompts per policy: The judge receives the original prompt, candidate output, policy text, and examples. It returns structured findings with severity and evidence.

2. Require evidence for every finding: The judge should cite the exact output span that triggered the issue. This keeps the decision reviewable.

3. Add confidence and disagreement: Run two judge passes for high-risk outputs. If judges disagree, route to human review instead of pretending certainty.

Phase 4: Build Rewrite and Block Flows (Day 7-10)

1. Implement safe rewrites: For fixable issues, rewrite only the problematic parts while preserving the useful answer. Examples: remove PII, soften overconfident claims, add uncertainty, or repair JSON.

2. Block non-fixable outputs: For serious safety issues, return a structured block response with reason codes. Do not expose internal policy text unnecessarily.

3. Log every decision: Store input hash, output hash, policy version, findings, action taken, latency, and final output. Auditability is the point.

Phase 5: Build the Review Dashboard (Day 10-12)

1. Create an audit queue: Human reviewers can inspect blocked or uncertain decisions, approve rewrites, override false positives, and leave notes.

2. Track policy performance: Show block rate, rewrite rate, false-positive rate, average latency, and most common violation types.

3. Add policy version comparison: When a policy changes, show how decisions would differ on historical examples. This prevents policy updates from causing chaos.

Phase 6: Polish for Portfolio (Day 12-14)

1. Demo multiple outcomes: Show an approved output, a rewritten output, a blocked output, and a human-review case.

2. Write the narrative: Position it as "a policy enforcement layer for LLM outputs," not a moderation toy. Put audit logs and decision transparency near the top.

Project 5: Production Log-to-Eval Dataset Builder

Topic: Evals, Data Flywheel, LLMOps, Human Review

What You're Building: A pipeline that mines production-like LLM logs, finds useful examples, converts them into evaluation cases, labels them with expected behavior or rubrics, and sends uncertain cases to a human review queue.

Why This Project Lands Interviews
The hard part of evals is often the dataset. Here you build a pipeline that turns real interactions into test cases, with review steps so the dataset does not rot.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Data processing and AI tooling
Log Store	PostgreSQL or DuckDB	Simple querying and local reproducibility
Embeddings	sentence-transformers / OpenAI embeddings	Clustering and deduplication
Clustering	HDBSCAN or scikit-learn	Finds interaction patterns
Labeling	LLM-as-judge + custom rubrics	Creates useful eval labels
Review UI	Streamlit	Fast human-in-the-loop workflow
Scheduler	Celery beat or cron	Nightly processing pipeline
Eval Format	JSONL	Easy to version and run in CI
Containerization	Docker Compose	Full local pipeline

Step-by-Step Build Guide

Phase 1: Design the Log Schema (Day 1-2)

- 1. Define the unified log format:** Capture prompt, system prompt, response, model, feature name, latency, token counts, user feedback, retry count, error status, and timestamp.
- 2. Add privacy controls:** Build redaction rules for emails, phone numbers, secrets, and names. Store both redaction status and redaction method.
- 3. Seed synthetic logs:** Create 1,000 simulated logs across a few features. Include good responses, bad responses, user retries, malformed outputs, and safety edge cases.

Phase 2: Sample and Classify Interactions (Day 2-5)

- 1. Build sampling modes:** Implement random sampling, failure-biased sampling, and diversity sampling. Failure-biased sampling should over-select logs with negative feedback, retries, or errors.

2. Cluster prompts: Embed prompts and cluster them to discover common categories. Give each cluster a human-readable label using representative examples.

3. Identify high-value candidates: Prioritize unusual prompts, low-quality outputs, high-impact features, edge cases, and clusters with poor eval coverage.

Interview Talking Point
Explain that a good eval set should not mirror traffic perfectly. It should over-represent risk, failures, and edge cases because those are where regressions hurt.

Phase 3: Auto-Generate Eval Labels (Day 5-8)

1. Decide eval type per example: Some examples need a golden answer. Others need a rubric. Some need expected refusal. Pick the label type based on the interaction.

2. Generate labels with confidence: Use a strong model to propose expected behavior, key assertions, forbidden assertions, and scoring rubric. Run multiple passes for important examples.

3. Deduplicate aggressively: Compare against existing eval cases and skip near-duplicates. Track why each candidate was accepted or rejected.

Phase 4: Build Human Review (Day 8-10)

1. Create a review queue: Low-confidence labels go to reviewers. Show the original interaction, proposed labels, similar existing cases, and quick approve/edit/reject actions.

2. Track reviewer edits: Store what changed and why. Use this to improve labeling prompts and measure auto-label quality.

3. Add dataset status: Every eval case should be draft, approved, rejected, or deprecated. This prevents messy datasets.

Phase 5: Connect to an Eval Runner (Day 10-12)

1. Export approved cases to JSONL: Keep the output format simple: input, expected behavior, rubric, tags, difficulty, source cluster, and date added.

2. Run nightly evals: Execute the growing dataset against a model endpoint and compare performance to the previous run.

3. Track dataset health: Show total cases, cases by category, cases by difficulty, freshness, auto-labeled percentage, and human-reviewed percentage.

Phase 6: Polish for Portfolio (Day 12-14)

1. Show the flywheel: Demo logs entering the system, candidates being selected, labels being generated, humans approving, and the eval dataset growing.

2. Use dataset numbers: Example: "Generated 300 approved eval cases across 12 categories from 5,000 simulated production logs, with 82% auto-label acceptance after review."

Project 6: Permissioned Tool-Using Agent Sandbox

Topic: Agents, Tool Use, Security, Human-in-the-Loop, Observability

What You're Building: A controlled agent environment where an LLM can use tools like file search, calculators, API calls, and database queries - but every tool has permissions, rate limits, audit logs, and human approval for sensitive actions.

Why This Project Lands Interviews
Instead of a flashy agent demo, this one focuses on permissions, approvals, tool logs, and clear failure handling. Those are the parts teams worry about.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Agent and API ecosystem
Orchestration	LangGraph	Explicit state machine for agent workflows
Tool Schemas	Pydantic	Validates tool inputs and outputs
API	FastAPI	Serves tasks and approval actions
Queue	Redis + Celery	Handles async tool execution

Component	Tool / Library	Why This Choice
Storage	PostgreSQL	Audit logs, permissions, task state
UI	Streamlit or React	Human approval and trace view
Observability	OpenTelemetry	Traces agent decisions and tool calls
Containerization	Docker Compose	Runs services locally

Step-by-Step Build Guide

Phase 1: Design the Agent and Tool Model (Day 1-3)

- 1. Define the agent role:** Build a "workspace assistant" that can answer questions, inspect files, summarize data, call safe APIs, and prepare actions for approval.
- 2. Create a tool registry:** Every tool has name, description, input schema, output schema, allowed roles, rate limit, risk level, and whether approval is required.
- 3. Build safe starter tools:** Add calculator, file reader over a sandbox folder, web-search stub or mock search, CSV query tool, and ticket creation mock API.

Phase 2: Build Permission Checks (Day 3-5)

- 1. Add user and role permissions:** Users have roles like viewer, analyst, operator, and admin. Tools check permissions before execution.
- 2. Add risk levels:** Low-risk tools execute immediately. Medium-risk tools require confirmation. High-risk tools require human approval before execution.
- 3. Block invalid tool inputs:** Validate every tool call with Pydantic. Never let raw model text become a command without validation.

Interview Talking Point
The key phrase is "the model proposes, the system disposes." The LLM suggests actions, but deterministic permission checks decide what happens.

Phase 3: Build the LangGraph Workflow (Day 5-8)

1. **Create graph nodes:** Intake, plan, tool selection, permission check, tool execution, result reflection, approval wait, final response.
2. **Add conditional routing:** If permission fails, return a safe explanation. If approval is needed, pause the task. If a tool fails, let the agent retry with a safer alternative.
3. **Store task state:** Persist every step so a paused task can resume after human approval.

Phase 4: Build Human Approval (Day 8-10)

1. **Create an approval queue:** Show proposed action, tool name, arguments, risk level, model reasoning summary, and expected effect.
2. **Add approve / reject / modify:** A human can approve as-is, edit the tool arguments, reject, or ask the agent to re-plan.
3. **Log decisions:** Store who approved, what changed, and why. This keeps the agent auditable.

Phase 5: Build Observability (Day 10-12)

1. **Trace every decision:** Capture prompts, chosen tools, permission decisions, tool outputs, retries, approval events, latency, and cost.
2. **Build a trace viewer:** Show the workflow as a timeline. Clicking a step reveals inputs, outputs, and decision reasons.
3. **Add safety analytics:** Track tool usage, blocked attempts, approval rate, rejected actions, and most common failure reasons.

Phase 6: Polish for Portfolio (Day 12-14)

1. **Demo a safe and unsafe task:** Show the agent completing a low-risk analysis, then attempting a sensitive action that gets routed to approval.
2. **Write the architecture narrative:** Focus on permission boundaries, audit logs, and human-in-the-loop design. Those details make the project feel production-minded.

Project 7: RAG Freshness and Drift Monitor

Topic: RAG, Data Quality, Monitoring, Knowledge Drift

What You're Building: A monitoring system that watches a RAG knowledge base for stale documents, changed source content, retrieval drift, missing coverage, and outdated answers. It tells the team when the index needs rebuilding or when answers are becoming less trustworthy.

Why This Project Lands Interviews
RAG does not end after indexing. This build covers the maintenance work: stale sources, retrieval drift, answer drift, and re-index triggers.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Data pipelines and AI tooling
Source Watcher	Git diff / filesystem watcher	Detects content changes
Embeddings	OpenAI or sentence-transformers	Measures semantic change
Vector Store	Qdrant / ChromaDB	Stores indexed chunks with metadata
Scheduler	Cron / Celery beat	Runs checks regularly
Eval Runner	Custom RAG evals	Tracks answer and retrieval drift
Storage	SQLite or PostgreSQL	Change history and metrics
Dashboard	Streamlit	Freshness and drift visibility
Containerization	Docker	Reproducible local monitoring stack

Step-by-Step Build Guide

Phase 1: Build a Baseline RAG Index (Day 1-3)

1. **Create a small knowledge base:** Use internal-doc-style Markdown files: policies, product pages, troubleshooting docs, and changelogs.
2. **Index documents with metadata:** Store document version, last modified date, source path, section heading, chunk hash, and embedding version.
3. **Save an index manifest:** The manifest records what was indexed, when, with which embedding model, and with which chunking strategy.

Phase 2: Detect Source-Level Freshness Issues (Day 3-5)

1. **Track content changes:** Compare current document hashes to the manifest. Detect added, removed, and modified sections.
2. **Score semantic change:** A one-word typo should not trigger panic. Compute semantic similarity between old and new sections to estimate how meaningful the change is.
3. **Prioritize re-indexing:** Flag high-impact changes first: pricing, policy, API behavior, troubleshooting steps, and security instructions.

Interview Talking Point
Explain that not all document changes are equal. A production system should prioritize re-indexing based on user impact instead of file modification time alone.

Phase 3: Detect Retrieval Drift (Day 5-8)

1. **Build a probe question set:** Create 50 recurring questions tied to known source sections.
2. **Run probes against old and new indexes:** Compare which chunks are retrieved and whether the top result changes after document updates.
3. **Flag suspicious changes:** If a known question no longer retrieves the expected section, mark it as retrieval drift.

Phase 4: Detect Answer Drift (Day 8-10)

1. **Generate answers for probe questions:** Run the same questions through the RAG pipeline over time.

2. Compare answers semantically: Use LLM-as-judge to identify whether the answer meaning changed, whether the change was expected, and whether citations still support the answer.

3. Track stale answer risk: If source docs changed but generated answers did not, the system may be serving stale knowledge. Flag this clearly.

Phase 5: Build Alerts and Dashboard (Day 10-12)

1. Build freshness scorecards: Show docs changed, chunks stale, probes drifting, answer drift, and re-index recommendations.

2. Add alerts: Send Slack notifications when high-risk docs changed without re-indexing, or when probe questions fail.

3. Add one-click rebuild command: From the dashboard, trigger re-indexing for affected docs and rerun probe tests.

Phase 6: Polish for Portfolio (Day 12-14)

1. Demo a stale doc scenario: Change a policy document, show the system detecting risk, rebuild the index, and show probes returning to healthy.

2. Write the narrative: Frame it as "monitoring for knowledge freshness in RAG systems." Most candidates do not have this lifecycle angle.

Project 8: Natural Language to API Assistant

Topic: Tool Calling, Guardrails, OpenAPI, Workflow Automation

What You're Building: A natural language interface that turns user requests into safe API calls against a mock business system. It reads an OpenAPI schema, plans the correct endpoint calls, validates parameters, runs dry-runs for risky actions, and asks for confirmation before making changes.

Why This Project Lands Interviews
Tool calling is useful only when the model respects the API contract. This build gives you a strong story around schema-aware planning, dry runs, confirmation, and safe execution.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Fast API and schema tooling
API Framework	FastAPI	Build mock business APIs and assistant API
API Schema	OpenAPI / JSON Schema	Realistic tool contract
LLM	GPT-4o-mini / Claude Haiku	Function planning and parameter extraction
Validation	Pydantic + jsonschema	Prevents invalid tool calls
Storage	PostgreSQL or SQLite	Request logs and approval state
UI	Streamlit	Demonstrates natural language workflows
Testing	pytest + VCR-style mocks	Reliable tool-call tests
Containerization	Docker Compose	Assistant + mock API stack

Step-by-Step Build Guide

Phase 1: Build the Mock Business API (Day 1-3)

- 1. Create a realistic domain:** Use a mock SaaS admin system with customers, subscriptions, invoices, tickets, and refunds.
- 2. Expose OpenAPI docs:** Build endpoints for read actions and write actions. Examples: get customer, list invoices, create ticket, update plan, issue refund.
- 3. Add permission metadata:** Mark each endpoint as read-only, low-risk write, or high-risk write. Include required roles.

Phase 2: Build Schema-Aware Planning (Day 3-5)

- 1. Parse the OpenAPI schema:** Extract endpoint names, descriptions, parameters, request bodies, response schemas, and risk tags.

2. Select relevant endpoints: Given a user request, retrieve candidate endpoints from the schema using embeddings or keyword search.

3. Generate a call plan: The LLM outputs a structured plan: endpoint, method, parameters, reason, expected result, and whether confirmation is required.

Interview Talking Point
Schema-aware tool use is safer than giving the model a vague list of tools. The schema gives the system a contract it can validate.

Phase 3: Build Validation and Dry Runs (Day 5-8)

1. Validate parameters: Use JSON Schema and Pydantic before any API call. Reject missing, ambiguous, or invalid fields.

2. Add dry-run mode: For write actions, show what would happen without changing data. The assistant should explain the planned action in plain English.

3. Ask for confirmation: High-risk actions require explicit approval. Store the pending plan and resume only after confirmation.

Phase 4: Execute Multi-Step Workflows (Day 8-10)

1. Support chained calls: Example: find customer by email, fetch subscription, check invoice status, then create a support ticket.

2. Pass outputs between steps: Store intermediate results in a typed workflow state. Do not rely on unstructured memory.

3. Handle failures gracefully: If an API call fails or returns multiple matches, ask a clarifying question instead of guessing.

Phase 5: Build Logs, UI, and Tests (Day 10-12)

1. Log every plan and call: Capture user request, selected endpoints, parameters, validation result, approval status, API response, and final answer.

2. Build a workflow UI: Show the user request, planned calls, dry-run preview, approval button, and final result.

3. Create a golden workflow test suite: Write 40-50 natural language requests with expected endpoint plans. Include ambiguous and unsafe requests.

Phase 6: Polish for Portfolio (Day 12-14)

- 1. Demo a full workflow:** Show a read-only request, a multi-step workflow, and a high-risk write that requires approval.
- 2. Write the narrative:** Lead with "safe tool use over real API contracts." That phrase maps closely to the work many AI platform teams are doing.

Project 9: Multimodal Document Intake Reviewer

Topic: Multimodal AI, OCR, Structured Extraction, Human-in-the-Loop

What You're Building: A document intake system that accepts scanned forms, PDFs, screenshots, and images; extracts structured fields; validates them against business rules; and sends low-confidence results to a human review queue.

Why This Project Lands Interviews
Document intake is a common enterprise use case. This build combines OCR, vision fallback, structured extraction, validation, and human review in one practical workflow.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Strong data and AI ecosystem
OCR	Tesseract + EasyOCR	Open-source OCR baseline
PDF Parsing	PyMuPDF	Extracts text and page images
Vision LLM	GPT-4o vision / Claude vision	Handles poor scans and complex layouts
Structured Output	Pydantic + instructor	Reliable extraction schemas
Queue	Celery + Redis	Async document processing

Component	Tool / Library	Why This Choice
Storage	PostgreSQL + local object storage	Metadata and original files
Review UI	Streamlit or React	Human correction workflow
Containerization	Docker Compose	Multi-service local demo

Step-by-Step Build Guide

Phase 1: Build Document Upload and Preprocessing (Day 1-3)

- 1. Accept multiple formats:** Support PDFs, PNGs, JPEGs, and TIFFs. Store the original file and a normalized page image for each page.
- 2. Detect document type:** Classify documents like invoice, reimbursement receipt, insurance claim, onboarding form, or contract summary.
- 3. Preprocess images:** Add rotation correction, contrast enhancement, noise removal, and resolution checks. Track preprocessing steps in metadata.

Phase 2: Build OCR and Vision Fallback (Day 3-5)

- 1. Run OCR first:** Extract text with Tesseract or EasyOCR. Store text by page and line if possible.
- 2. Estimate OCR confidence:** Use OCR confidence scores, text density, and format sanity checks to decide whether OCR is usable.
- 3. Use vision fallback:** If OCR confidence is low, send the page image to a vision-capable model and ask it to preserve layout.

Interview Talking Point
Explain that vision models are powerful but expensive. A production design should use them as fallback, not as the default for every page.

Phase 3: Extract Structured Data (Day 5-8)

- 1. Define schemas by document type:** For invoices, extract vendor, invoice number, dates, line items, tax, and total. For forms, extract applicant details and required fields.
- 2. Use structured output:** The LLM should return data that validates against Pydantic. Include field-level source references where each value came from.
- 3. Handle long documents:** Process by page or section and merge outputs. If two pages disagree, flag the conflict instead of hiding it.

Phase 4: Validate and Route (Day 8-10)

- 1. Add type validation:** Dates parse, totals are numbers, required fields exist, and enums are valid.
- 2. Add business rules:** Invoice totals must equal line items plus tax. Claim dates must be within policy windows. Vendor names must match known vendors.
- 3. Route by confidence:** High confidence goes auto-approved. Medium and low confidence go to review with reasons.

Phase 5: Build Human Review and Analytics (Day 10-12)

- 1. Create side-by-side review:** Show the document image and extracted fields. Clicking a field highlights its source area or page.
- 2. Let reviewers correct fields:** Store original value, corrected value, reviewer, and correction reason.
- 3. Track accuracy by field:** Show which fields fail most often, average review time, and auto-approval rate.

Phase 6: Polish for Portfolio (Day 12-14)

- 1. Demo a messy document:** Use a slightly rotated scan or screenshot where OCR is imperfect. Show extraction, validation, review, and correction.
- 2. Use operational metrics:** Example: "Auto-approved 64% of sample documents while routing low-confidence fields to review."

Project 10: Fine-Tune vs RAG Decision Lab

Topic: MLOps, Fine-Tuning, RAG, Benchmarking, Model Selection

What You're Building: An experiment framework that compares three approaches for a domain-specific AI task: prompt-only, RAG, and LoRA fine-tuning. It evaluates quality, cost, latency, maintainability, and failure modes so you can justify which approach is best.

Why This Project Lands Interviews
Fine-tuning sounds impressive, but only if you can justify it. This build compares prompt-only, RAG, and LoRA with the same benchmark so the decision is evidence-based.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Standard ML and eval tooling
Base Model	Llama / Mistral / Phi	Practical open-source options
Fine-Tuning	Hugging Face PEFT + TRL	Common LoRA workflow
RAG	ChromaDB / Qdrant	Retrieval baseline
Training Efficiency	QLoRA / Unsloth	Runs on limited GPU resources
Experiment Tracking	MLflow or Weights & Biases	Reproducibility and comparison
Evals	Custom benchmark + LLM judge	Task-specific measurement
Serving	vLLM / Ollama	Local inference and demo
Containerization	Docker	Reproducible pipeline

Step-by-Step Build Guide

Phase 1: Choose a Narrow Domain Task (Day 1-3)

1. **Pick a measurable task:** Good examples: support macro selection, policy clause classification, bug report severity prediction, or domain-specific email rewrite.
2. **Build a dataset:** Create 500-1,500 examples with instruction, input, expected output, category, and difficulty. Clean duplicates and formatting issues.
3. **Create a benchmark:** Hold out 100 examples as a fixed test set. Add 30 hard edge cases by hand.

Phase 2: Build the Prompt-Only Baseline (Day 3-4)

1. **Write a strong prompt:** Include task instructions, output schema, and a few examples.
2. **Run baseline evals:** Measure accuracy, quality score, schema validity, latency, and cost.
3. **Store outputs:** Keep raw outputs so you can compare where each approach succeeds or fails.

Interview Talking Point
The baseline is not a formality. It is the only way to know whether fine-tuning was worth the effort.

Phase 3: Build the RAG Variant (Day 4-7)

1. **Create a small knowledge base:** Add policies, examples, guidelines, or domain references that can help the task.
2. **Retrieve context for each input:** Use embeddings and metadata filters to find the most relevant reference examples.
3. **Evaluate the RAG version:** Compare quality to the prompt-only baseline. Track retrieval failures separately from generation failures.

Phase 4: Build the LoRA Fine-Tuned Variant (Day 7-10)

1. **Prepare training data:** Format data for instruction tuning. Split train/validation/test cleanly.

2. Fine-tune with LoRA: Start with modest settings: rank 8 or 16, low learning rate, early stopping, and validation tracking.

3. Track experiments: Log hyperparameters, training curves, validation metrics, GPU memory, and selected checkpoint.

Phase 5: Compare All Three Approaches (Day 10-12)

1. Run the same benchmark: Evaluate prompt-only, RAG, and fine-tuned model on the same test set.

2. Compare beyond accuracy: Include latency, cost, setup complexity, update difficulty, failure modes, and operational risk.

3. Write a recommendation: For example: "RAG wins when knowledge changes often; LoRA wins when style and label consistency matter most."

Phase 6: Polish for Portfolio (Day 12-14)

1. Build a comparison dashboard: Show side-by-side outputs and metrics for all three approaches.

2. Write the decision memo: Make it read like an internal architecture decision record. Reviewers notice judgment, not only implementation.

Project 11: AI Trace Explorer for LLM Workflows

Topic: LLMOps, Observability, Tracing, Debugging, Evals

What You're Building: An observability tool for multi-step LLM workflows. It records every prompt, model response, tool call, retrieved context, validation result, latency, cost, and error, then displays the full execution trace so engineers can debug failures quickly.

Why This Project Lands Interviews
When a workflow breaks, you need traces, not guesses. This build helps you explain how to debug retrieval, prompts, tools, validation, and generation step by step.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	AI and observability tooling
Workflow	Custom pipeline or LangGraph	Creates multi-step traces
Tracing	OpenTelemetry + custom spans	Industry-standard observability pattern
Storage	PostgreSQL + JSON blobs	Queryable metadata and full trace payloads
LLM Provider	OpenAI / Anthropic / local	Flexible workflow testing
UI	Streamlit or React	Trace explorer and failure view
Metrics	Prometheus optional	Latency/cost/error aggregation
Containerization	Docker Compose	Reproducible stack

Step-by-Step Build Guide

Phase 1: Build a Multi-Step AI Workflow (Day 1-3)

- 1. Choose a workflow:** Example: intake a support ticket, classify it, retrieve relevant docs, draft a response, validate the response, and create an internal note.
- 2. Make every step typed:** Use Pydantic models for each step's input and output. Tracing is much easier when step boundaries are clean.
- 3. Add controlled failure cases:** Include bad retrieval, malformed JSON, low-confidence classification, and failed validation examples.

Phase 2: Instrument the Workflow (Day 3-6)

- 1. Create a trace ID per run:** Every workflow execution gets a unique trace ID and a list of spans.
- 2. Wrap each step in a tracing decorator:** Capture step name, input, output, prompt, model, token usage, latency, cost, status, and errors.

3. Capture artifacts: Store retrieved chunks, tool arguments, validation failures, and retry attempts as structured artifacts.

Interview Talking Point
A trace should not only say that the final answer was bad. It should show the exact intermediate decision that made it bad.

Phase 3: Add Failure Classification (Day 6-8)

- 1. Define failure categories:** Retrieval miss, prompt failure, schema failure, tool error, hallucination, unsupported citation, timeout, and policy violation.
- 2. Classify failed runs:** Use rules first, then an LLM judge for nuanced cases. Store both category and evidence.
- 3. Add root-cause hints:** Suggest likely fixes: adjust chunking, add eval case, tighten schema, change prompt, or add retry.

Phase 4: Build the Trace Explorer UI (Day 8-11)

- 1. Show a timeline:** Display each step in order with status, latency, cost, and short summary.
- 2. Add expandable details:** Clicking a step reveals input, output, prompt, raw response, and artifacts.
- 3. Add diff views:** Compare successful and failed runs of the same workflow to spot where behavior diverged.

Phase 5: Build Metrics and Feedback Loop (Day 11-13)

- 1. Aggregate reliability metrics:** Track failure rate by step, average cost per workflow, P95 latency, retry rate, and most common failure category.
- 2. Turn failures into eval cases:** Let a user mark a trace as a regression and export it into your eval dataset.
- 3. Add alerts:** Trigger Slack alerts when a step's failure rate rises above threshold.

Phase 6: Polish for Portfolio (Day 13-14)

- 1. **Demo a failure diagnosis:** Run a workflow that produces a bad answer, open the trace, identify the broken step, and export it to evals.
- 2. **Write the narrative:** Frame it as "debugging infrastructure for AI workflows," which is much stronger than calling it a dashboard.

Project 12: Semantic Cache Gateway for LLM APIs

Topic: LLMOps, Cost Reduction, Latency Optimization, Embeddings

What You're Building: A drop-in gateway that detects semantically similar LLM requests, serves cached responses when safe, and tracks cost and latency savings. It supports exact cache keys, semantic similarity, TTLs, and feature-specific cache policies.

Why This Project Lands Interviews
Repeated LLM calls waste money and slow apps down. A semantic cache gives you a clear infrastructure project with cost, latency, and correctness tradeoffs.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	API and embedding ecosystem
Proxy	FastAPI	Drop-in API layer
Embeddings	OpenAI embeddings or sentence-transformers	Semantic similarity search
Cache Store	Redis + vector search or Qdrant	Fast lookup and metadata filtering
Provider Layer	OpenAI / Anthropic / Ollama	Provider-agnostic design

Component	Tool / Library	Why This Choice
Metrics	Prometheus	Cache hit and latency tracking
Dashboard	Grafana or Streamlit	Shows savings clearly
Load Testing	Locust or k6	Realistic workload simulation
Containerization	Docker Compose	Runs cache + dashboard locally

Step-by-Step Build Guide

Phase 1: Build the Proxy Contract (Day 1-3)

- 1. Mirror a chat completion API:** Accept model, messages, temperature, max tokens, and metadata. Return the provider-style response plus cache metadata.
- 2. Normalize requests:** Hash system prompt, model, temperature, tool settings, and feature name. Two prompts are only cache-compatible if these settings match.
- 3. Forward cache misses:** If no cache hit exists, call the provider, store the response, and return it.

Phase 2: Build Semantic Matching (Day 3-5)

- 1. Embed user intent:** Create embeddings for the user message or normalized prompt. Store embedding with response metadata.
- 2. Query nearest neighbors:** If similarity is above threshold, return cached response. Start conservative around 0.95.
- 3. Store debug info:** Save original prompt, matched prompt, similarity score, cache decision, and response ID.

Interview Talking Point

The hard part is not caching. The hard part is deciding when two requests are "similar enough" without returning a wrong answer.

Phase 3: Build Cache Policies (Day 5-8)

1. **Add TTLs by feature:** Stable FAQs can have long TTLs. Time-sensitive or user-specific requests should have short TTLs or caching disabled.
2. **Add policy tags:** Support tags like `user_specific`, `current_events`, `legal_sensitive`, and `creative`. Use tags to change threshold and TTL.
3. **Add invalidation endpoints:** Invalidate by model, system prompt hash, feature, user, tag, or time range.

Phase 4: Add Safety Checks (Day 8-10)

1. **Prevent cross-user leakage:** Never serve cached responses across users for personal or private contexts unless explicitly allowed.
2. **Add semantic hit validation:** For borderline matches, use a lightweight judge to confirm that the cached answer still fits the new prompt.
3. **Track near misses:** Log prompts just below threshold. These help tune cache policies.

Phase 5: Build Metrics and Load Test (Day 10-12)

1. **Export metrics:** Track hit rate, semantic hit rate, exact hit rate, average latency saved, cost saved, and wrong-hit reports.
2. **Build a dashboard:** Show real-time hit rate, latency comparison, cost savings, cache size, and threshold tradeoffs.
3. **Run a workload simulation:** Send 2,000 requests with repeated and paraphrased questions. Measure savings and false-hit rate.

Phase 6: Polish for Portfolio (Day 12-14)

1. **Demo instant responses:** Show a miss, an exact hit, a semantic hit, and an intentionally blocked unsafe cache hit.
2. **Use ROI:** Example headline: "Reduced simulated LLM cost by 41% and P95 latency by 68% with conservative semantic caching."

Project 13: AI Feature Rollout Monitor

Topic: LLMOps, Feature Flags, Canary Rollouts, Quality Monitoring, Rollback

What You're Building: A feature rollout system for AI features where new prompts, models, or RAG configurations are released gradually. The system monitors quality during rollout and automatically pauses or rolls back if performance drops.

Why This Project Lands Interviews
AI rollouts need more than an on/off switch. This build adds quality checks, canary stages, automatic pause, and rollback logic.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	AI and backend ecosystem
Flag Service	FastAPI	Central rollout API
Storage	PostgreSQL	Flags, assignments, metrics, audit logs
Cache	Redis	Fast flag evaluation and counters
Eval Worker	Celery	Async quality checks
Quality Metrics	LLM judge + user feedback	Measures output quality after serving
Dashboard	Streamlit or React	Rollout monitoring and controls
Alerting	Slack webhook	Rollback and pause notifications
Containerization	Docker Compose	Full local demo

Step-by-Step Build Guide

Phase 1: Build the Flag Evaluation SDK (Day 1-3)

1. **Define AI flag schema:** Include flag name, baseline config, candidate config, rollout percentage, quality threshold, owner, and rollback rule.
2. **Build consistent assignment:** Use hashing so the same user or session consistently gets the same variant during rollout.
3. **Add safe defaults:** If the flag service is down, the SDK returns the baseline config.

Phase 2: Build Quality Collection (Day 3-6)

1. **Log variant outputs:** Store which variant served each request, the output, latency, cost, and any user feedback.
2. **Score outputs asynchronously:** Use an LLM judge or task-specific metric after the response is returned. Do not add latency to the user path.
3. **Compare candidate to baseline:** Track rolling averages, P10 quality, error rate, and latency for both variants.

Interview Talking Point
AI rollouts should look at the bottom tail of quality, not only the average. A few terrible outputs can matter more than a slightly higher mean score.

Phase 3: Build Rollout Automation (Day 6-9)

1. **Define rollout stages:** Example: 1%, 5%, 25%, 50%, 100%. Each stage has minimum sample size and quality requirements.
2. **Auto-advance safely:** Move to the next stage only when metrics are healthy and enough samples have been collected.
3. **Pause or rollback automatically:** If quality drops, latency spikes, or safety issues appear, pause rollout or set traffic back to 0%.

Phase 4: Build Dashboard and Controls (Day 9-11)

1. **Show active rollouts:** Include current percentage, stage, health status, quality trend, and next checkpoint.

2. Add manual controls: Pause, resume, rollback, and promote to 100%. Require a reason for every action.

3. Show audit history: Track who changed rollout settings and why.

Phase 5: Test with a Demo AI Feature (Day 11-13)

1. Build a small AI feature: Use subject line generation, support summary, or product description rewrite.

2. Create good and bad variants: The bad variant should fail quality checks clearly so rollback is easy to demo.

3. Run the rollout simulation: Show the system advancing a good variant and stopping a bad one.

Phase 6: Polish for Portfolio (Day 13-14)

1. Record the rollout lifecycle: Show flag creation, staged rollout, quality monitoring, automatic pause, and rollback alert.

2. Write the narrative: Use the phrase "quality-aware feature flags for AI systems." It communicates the project quickly.

Project 14: Evaluation-as-a-Service Platform

Topic: Evals, LLM-as-Judge, Quality Gates, Judge Calibration

What You're Building: A centralized evaluation API that other AI features can call to score model outputs against rubrics. It supports custom metrics, judge calibration, batch evaluation, historical tracking, and release quality gates.

Why This Project Lands Interviews
Reusable eval infrastructure is much stronger than a one-off scoring script. This build covers rubrics, judge calibration, batch scoring, history, and quality gates.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Evaluation and API tooling
API	FastAPI	Clean service interface
Judge Models	OpenAI / Anthropic / local	Compare judge behavior
Structured Output	Pydantic + instructor	Reliable scoring objects
Storage	PostgreSQL	Runs, scores, rubrics, calibration sets
Queue	Celery + Redis	Batch evals and async scoring
Dashboard	Streamlit	Explore runs and score distributions
Stats	scipy / numpy	Agreement and confidence intervals
Containerization	Docker Compose	Service + worker + dashboard

Step-by-Step Build Guide

Phase 1: Design the Eval API (Day 1-3)

- 1. Define evaluation requests:** Input includes prompt, model output, optional reference answer, rubric, task type, and metadata.
- 2. Define score output:** Return overall score, dimension scores, pass/fail, explanation, evidence spans, judge confidence, and policy warnings.
- 3. Support reusable rubrics:** Store rubrics for summarization, classification, RAG faithfulness, tone, completeness, and safety.

Phase 2: Build Judge-Based Scoring (Day 3-6)

- 1. Write dimension-specific judge prompts:** Each rubric dimension should have clear scoring rules and examples.

2. Enforce structured results: Use Pydantic so every score has the same shape. Invalid judge output should retry or fail cleanly.

3. Add evidence requirements: The judge must explain what part of the output caused each score.

Interview Talking Point
LLM-as-judge is useful, but it is not magic. You need calibration examples, consistency checks, and human review for high-risk decisions.

Phase 3: Calibrate the Judge (Day 6-8)

1. Build a calibration set: Create 50 examples with human scores. Include excellent, mediocre, and bad outputs.

2. Compare judge to human labels: Measure agreement, average score difference, and where the judge is too harsh or too generous.

3. Tune prompts and thresholds: Adjust rubrics and pass/fail thresholds based on calibration results.

Phase 4: Build Batch Evals and Quality Gates (Day 8-11)

1. Add batch endpoint: Accept a list of outputs and run evals asynchronously.

2. Generate run reports: Show score distributions, failed cases, dimension-level weakness, and examples of judge reasoning.

3. Add release gates: A CI job can call the eval service and fail if pass rate, safety score, or faithfulness score drops below threshold.

Phase 5: Build Dashboard and Analytics (Day 11-13)

1. Create a run explorer: Compare eval runs over time and drill into failed examples.

2. Track judge drift: If judge model or rubric changes, rerun calibration examples and compare scores.

3. Add metric health: Show score variance, disagreement between judges, and human override rate.

Phase 6: Polish for Portfolio (Day 13-14)

- 1. **Demo it across two features:** Score a RAG answer and a summarization output using different rubrics.
- 2. **Write the case study:** Position it as "shared evaluation infrastructure for AI platform teams." Start with calibration results.

Project 15: Local LLM Deployment Kit

Topic: MLOps, Inference, Local Models, Quantization, Performance Testing

What You're Building: A deployment kit that runs an open-source LLM locally or on a small server, exposes an OpenAI-compatible API, benchmarks latency and throughput across quantization levels, and includes monitoring, rollback, and model comparison reports.

Why This Project Lands Interviews
Local deployment is useful when teams care about cost, data control, or vendor flexibility. This build covers serving, quantization, benchmarking, monitoring, and rollback.

Tech Stack

Component	Tool / Library	Why This Choice
Language	Python 3.11+	Benchmark scripts and API wrappers
Serving	vLLM, Ollama, or llama.cpp	Practical local inference options
Models	Llama / Mistral / Qwen / Phi	Realistic open-source choices
Quantization	GGUF / AWQ / GPTQ	Tests cost-performance tradeoffs
API	FastAPI wrapper or OpenAI-compatible server	Easy app integration

Component	Tool / Library	Why This Choice
Benchmarking	Locust / custom async runner	Measures throughput and latency
Metrics	Prometheus	Production-style monitoring
Dashboard	Grafana	Visual performance monitoring
Containerization	Docker + NVIDIA runtime optional	Reproducible deployment path

Step-by-Step Build Guide

Phase 1: Pick Models and Serving Path (Day 1-3)

- 1. **Choose a target task:** Pick something measurable like summarization, classification, extraction, or support reply drafting.
- 2. **Select two or three model variants:** Compare a small model, a medium model, and a quantized model. Document expected tradeoffs.
- 3. **Set up serving:** Use Ollama for simplicity or vLLM for higher-throughput serving. Expose a chat completion endpoint.

Phase 2: Build the Benchmark Harness (Day 3-5)

- 1. **Create a request dataset:** Use 100-200 prompts with realistic input lengths and expected output lengths.
- 2. **Measure performance:** Track tokens per second, time to first token, P50/P95 latency, throughput, memory usage, and error rate.
- 3. **Test concurrency:** Run benchmarks at 1, 5, 10, and 25 concurrent users.

Interview Talking Point
Local models are not automatically cheaper or better. You have to measure hardware cost, latency, quality, and operational complexity together.

Phase 3: Build Quality Evaluation (Day 5-8)

1. **Score outputs:** Use task-specific metrics and LLM-as-judge against reference answers.
2. **Compare against a cloud baseline:** Run the same prompts against a hosted model and compare quality, cost estimate, and latency.
3. **Identify acceptable use cases:** Some tasks may work well locally; others may need stronger hosted models. Document this honestly.

Phase 4: Add Monitoring and Rollback (Day 8-10)

1. **Export metrics:** Latency, throughput, token rate, GPU memory, queue depth, errors, and model version.
2. **Build Grafana dashboards:** Show live inference health and historical benchmark results.
3. **Add model rollback:** Support switching active model versions through config. Log every switch.

Phase 5: Package the Deployment Kit (Day 10-12)

1. **Write Docker Compose files:** Include server, metrics, dashboard, and optional benchmark runner.
2. **Add config files:** Let users set model path, quantization, max context length, batching settings, and concurrency limits.
3. **Create setup scripts:** Make it easy to pull a model, start the server, run a benchmark, and generate a report.

Phase 6: Polish for Portfolio (Day 12-14)

1. **Publish benchmark results:** Include a table comparing models by quality, latency, throughput, memory, and cost estimate.
2. **Write the deployment memo:** Frame it as an internal recommendation: which model should be used for which workload, and why.

Final Portfolio Advice

Do not build these like isolated school assignments. Build them like internal tools a real AI team would use.

For each finished project, try to include:

- A clean architecture diagram.
- A reproducible Docker setup.
- A README written like internal engineering documentation.
- A small but intentional evaluation dataset.
- A dashboard or report with real numbers.
- A 3-5 minute demo video.
- A short case study that starts with the operational problem, not the model.

The best portfolio story is not "I used LangChain." It is: "I built a system that improved quality, reduced cost, caught regressions, made failures easier to debug, or made AI safer to ship."

Disclaimer: This guide is written with personal experiences, resources from the internet and structured with the help of AI. Highly useful for anyone who wants to build a career in AI